A Mini Project Report of

PCC-201-COM : Data Structures

Submitted by

DNYANESHWAR PATIL (ROLL NO. 61)

Under the Guidance of

Prof. Tejal H. Patil

Department of Computer Engineering

BHARATI VIDYAPEETH'S

COLLEGE OF ENGINEERING, LAVALE, PUNE, 412115

Savitribai Phule Pune University

[2026-27]

BHARATI VIDYAPEETH
COLLEGE OF ENGINEERING, LAVALE, PUNE

CERTIFICATE

This is to certify that the **"Mini Project Report"** submitted by **Dnyaneshwar Vikram Patil (61),** on **"Implementation of Maze Generation [DFS]"** is work done by him and submitted during **2026–2027** academic year, in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer Engineering, at **Bharati Vidyapeeth's College of Engineering, Lavale, Pune.**

Prof. Tejal H. Patil
(Guide)

Dr. Lalit V. Patil
Head of Computer Engg. Department

ACKNOWLEDGEMENT

It gives me immense pleasure to express my gratitude and thanks to all those who have helped in the timely completion of this Mini-Project work. I have great pleasure in expressing my deep sense of gratitude and indebtedness to **Prof. Tejal H. Patil**, my Guide, for her invaluable guidance, constant supervision and sustained encouragement throughout the tenure of this Mini Project work.

I would like to express my thanks to **Dr. Lalit V. Patil**, Head of Department, Department of Computer Engineering, Bharati Vidyapeeth's College of Engineering, Lavale, Pune for his constant support and encouragement.

I would like to express my thanks to **Prof. Dr. R. N. Patil,** Principal, Bharati Vidyapeeth's College of Engineering, Lavale, Pune for permitting me to take up the project work. I am thankful to my parents and friends for their kind help and support throughout the course.

Dnyaneshwar Vikram Patil
Student Name



ABSTRACT

Maze generation is the process of creating a maze with connected passages and walls. This project uses Depth-First Search (DFS) to generate a maze represented as a grid. Starting from a selected cell, the algorithm visits an unvisited neighbouring cell, removes the wall between the cells, and continues exploring. When it reaches a cell with no unvisited neighbours, it backtracks to an earlier cell and continues until all cells have been visited.

The maze is represented as a grid in which each cell has walls on its four sides. DFS uses a stack-like process: the current cell is marked visited, an unvisited neighbour is selected, the wall between them is removed, and the neighbour becomes the current cell. Backtracking ensures that the algorithm explores remaining branches and creates a connected maze.

TABLE OF CONTENTS

SR.NO.	CONTENT	PAGE NO.
	Acknowledgement	i
	Abstract	ii
	Content	iii
	List Of Figures	iv
1	Introduction	1
	a) Overview	
	b) Problem Statement	
	c) Objective	
2	Literature Survey	2
3	UML Diagram	3
4	System Requirement	4
5	System Design	5
	a) Existing system	
	b) Proposed system	
	c) Working Diagram	
	d) System code	
6	Result	8
7	Application	9
8	Future Scope	10
9	Conclusion	11
10	References	12

LIST OF FIGURES

Figure	Description	Page No.
Fig 3.1	UML Class Diagram	3
Fig 5.1	Working Diagram of Maze Generation using DFS	6
Fig 5.2	System Code	7
Fig 6.1	Result (Demo Program and Output)	8

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Literature Survey	2
6.1	Top Keywords Obtained from the Demo Text	8

1. INTRODUCTION

1.1 Overview:

The project “Implementation of Maze Generation [DFS]” generates a maze using Depth-First Search. The maze is represented as a grid of cells connected by passages. DFS explores one path as far as possible, removes walls between adjacent cells, and backtracks when no unvisited neighbour remains. This process continues until all cells are visited, producing a connected maze.

1.2 Problem Statement:

The problem is to generate a connected maze from a grid while ensuring that cells are reachable through passages. Manually designing walls and paths is time-consuming and does not scale well. The project applies Depth-First Search to explore the grid systematically and remove walls between neighbouring cells to form maze passages.

1.3 Objective:

The objective of this project is to design and implement a maze generation algorithm using Depth-First Search in Python. The project aims to:

- Represent the maze as a two-dimensional grid of cells.
- Use Depth-First Search to explore unvisited neighbouring cells.
- Remove walls between adjacent cells to create connected passages.
- Use backtracking to visit all reachable cells and generate a complete maze.

2. LITERATURE SURVEY

Graph traversal algorithms are used to explore connected structures and visit states systematically. Depth-First Search is especially useful for maze generation because it follows one branch until it cannot continue, then backtracks to explore other branches. This section reviews the concepts relevant to representing a maze as a grid and generating passages with DFS.

Table 2.1 summarizes the main references that form the theoretical background of this project.

Year	Paper Name	Author	Conclusion
2024	Analysis of Maze Generation Algorithms	J. Vijaya et al.	The study compares DFS, recursive backtracking, and Prim's algorithm to improve maze diversity and reduce predictable maze patterns.
2025	Maze Generation Algorithms: A Comparative Research with a Focus on Recursive Backtracking in OCaml	Jiahao Li	The study discusses several maze generation algorithms and highlights the effectiveness of recursive backtracking in generating connected mazes with unique paths.
2025	Comprehensive Comparison of Maze Generating and Solving Algorithms	Junlong Li	The study compares maze generation methods and evaluates DFS, BFS, and Wall-Follower algorithms based on performance, memory usage, and path quality.
2025	AMaze: An Intuitive Benchmark Generator for Fast Prototyping of Generalizable Agents	Kevin Godin-Dubois, Karine Miras, and Anna V. Kononova	The study presents a maze-generation framework for testing navigation agents in different maze environments and improving their ability to generalize.
2026	A Comparative Analysis of DFS, Eller's and Wilson's Algorithms for Maze Generation	Jiarui Kang	The study compares the algorithms in terms of time complexity, space complexity, and randomness, highlighting the simplicity of DFS and the different characteristics of alternative methods.

3. UML DIAGRAM

The UML class diagram for the Maze Generator represents the grid, cell walls, visited state, neighbour selection, wall removal and the DFS generation routine. A stack or recursive call sequence records the current exploration path and enables backtracking.

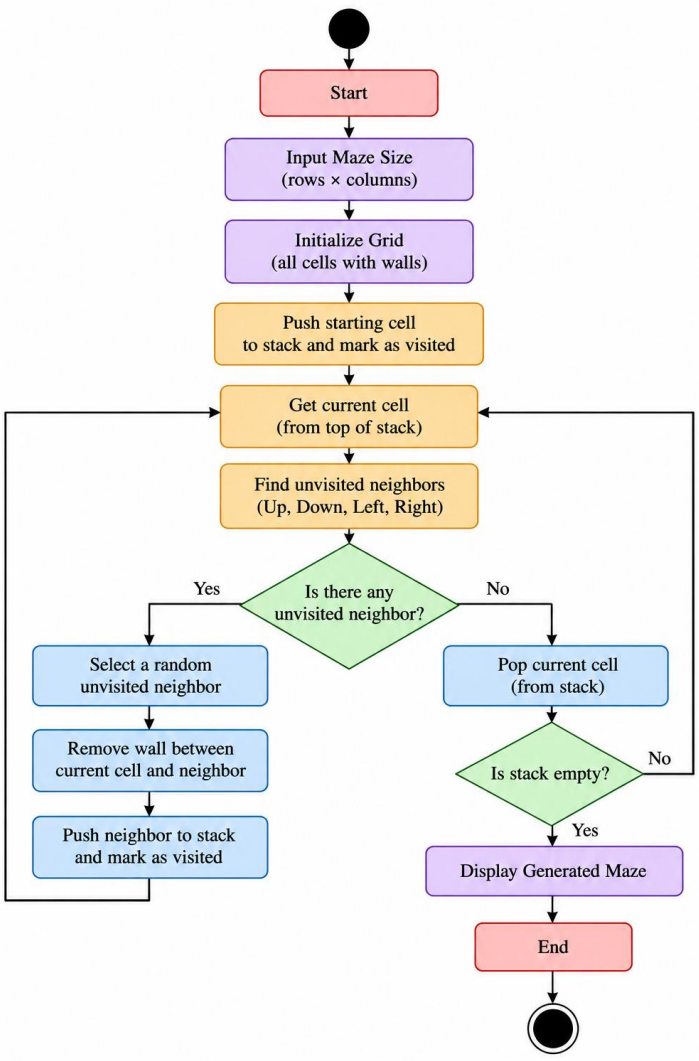


Fig 3.1: UML Class Diagram

4. SYSTEM REQUIREMENT

4.1 HARDWARE REQUIREMENTS

1. Operating System: Windows 11 64-bit
2. Processor Type: Intel Core
3. Recommended: Core 2 Duo or higher
4. Processor speed: 2.30 GHz or higher
5. RAM: Minimum 512 MB
6. Hard disk: 512 GB or more

4.2 SOFTWARE REQUIREMENTS

1. Jupyter Notebook (JupyterLab)
2. Language: Python 3 (ipykernel)
3. Python modules: re, collections

5. SYSTEM DESIGN

Existing System:

In a basic approach, the words of a text are stored in a list and, for every word, the list is searched to find whether the word already exists so that its count can be increased. This linear search needs O(n) time for every word, so counting a whole document takes $O(n^2)$ time in the worst case. Manual counting or spreadsheet formulas are also slow and error-prone for large texts, and they do not remove common words automatically.

Key approaches commonly used for maze generation:

Manual wall and passage design (time-consuming for large grids)

- ✓ Randomly removing walls (may create disconnected or cyclic layouts)
- ✓ Depth-First Search with backtracking (systematically visits cells)
- ✓ Visited-cell tracking and neighbour selection (avoid repeated visits)
- ✓ Library classes such as collections.Counter

Proposed System:

The proposed system represents each cell as a grid position with walls on its four sides. DFS starts at a selected cell, marks it visited, chooses a random unvisited neighbour, removes the wall between the current and neighbouring cells, and continues. If no unvisited neighbour remains, the algorithm backtracks until another unvisited neighbour is found. The process ends when all cells have been visited.

- ✓ Create a grid and mark every cell as unvisited.
- ✓ Choose a starting cell, mark it visited and push it onto the stack.
- ✓ Select a random unvisited neighbour and remove the shared wall.
- ✓ Continue exploring; backtrack when the current cell has no unvisited neighbours.
- ✓ Stop when all cells are visited and display the generated maze.

Time Complexity:

Operation	Time	Space
Tokenize text (n words)	O(n)	O(n)
Count with hash table	O(n) average	O(k)
Sort k unique words	O(k log k)	O(k)
top_n (count + sort + slice)	O(n + k log k)	O(k)

Working Diagram:

Fig 5.1 shows the working of the system. The grid is initialized, a starting cell is marked visited, and DFS repeatedly selects an unvisited neighbour and removes the wall between the two cells. When no unvisited neighbour remains, the algorithm backtracks until all cells have been explored.

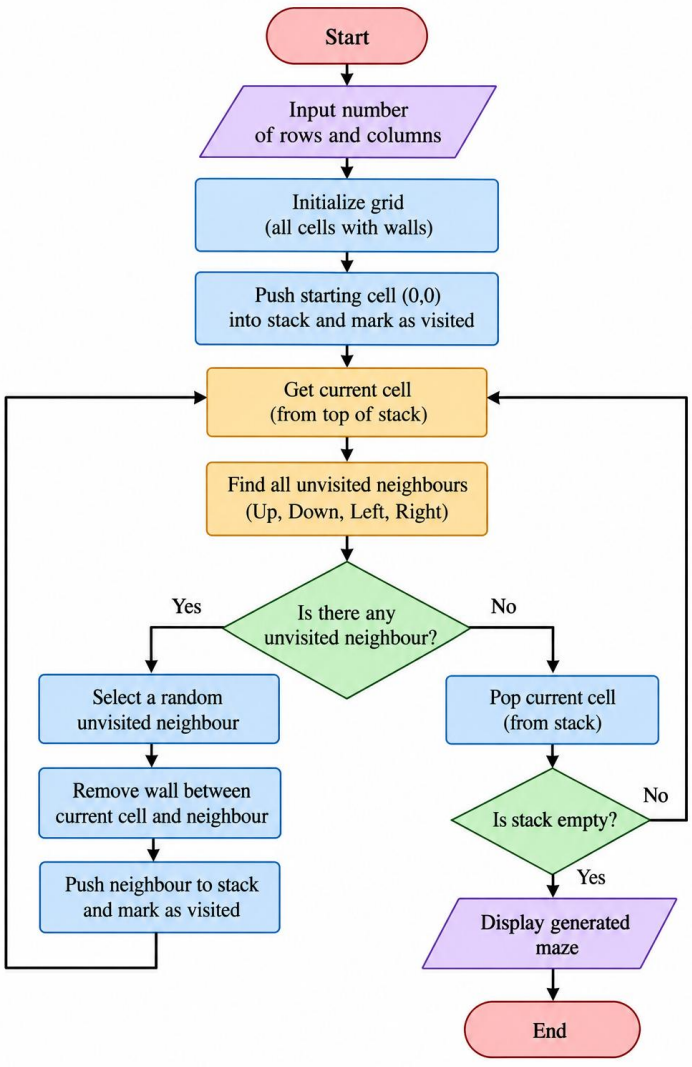


Fig 5.1: Working Diagram of Maze Generation using DFS

SYSTEM CODE

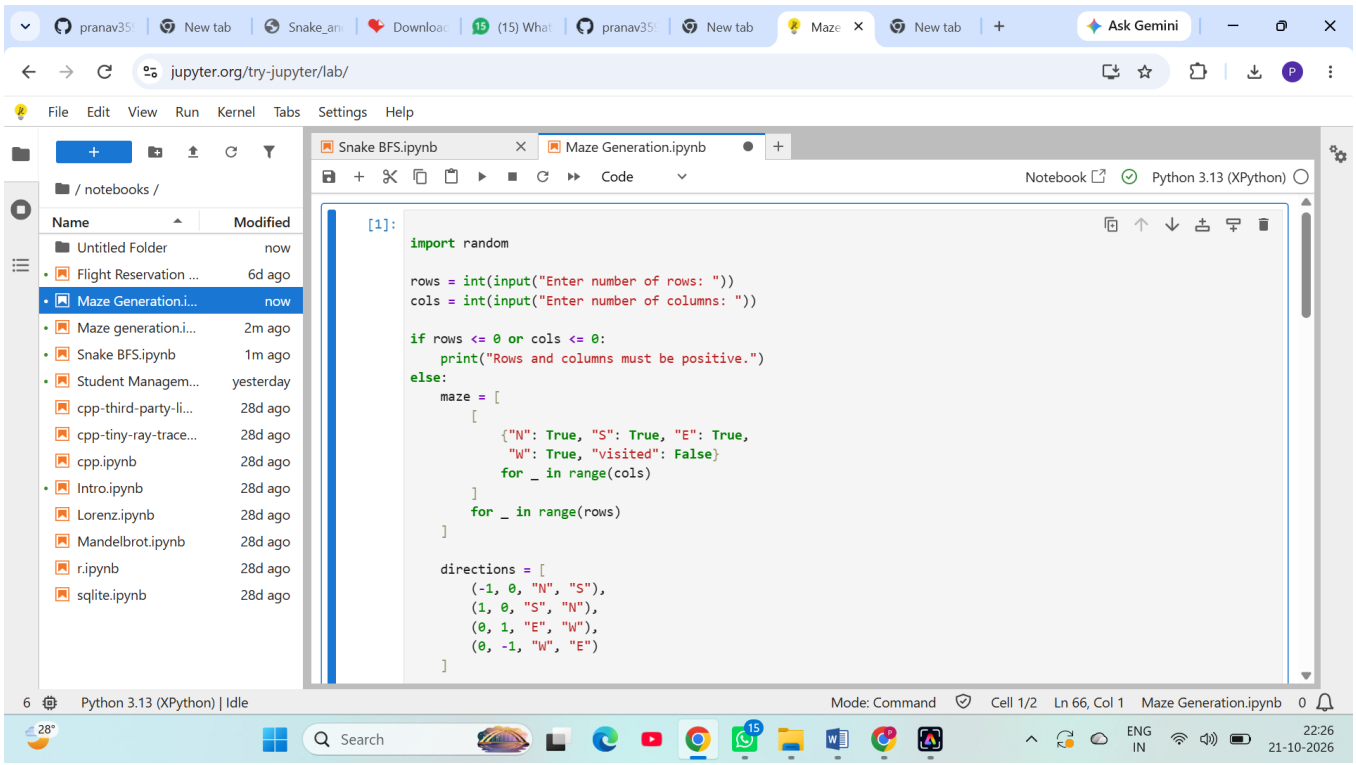


Fig 5.2(b): System code – Demo program

6. RESULT

The program generates a maze by visiting cells with Depth-First Search and removing walls between adjacent cells. The algorithm uses a visited set and backtracking stack to ensure that every cell is reached. The resulting maze is displayed as a grid of walls and passages.

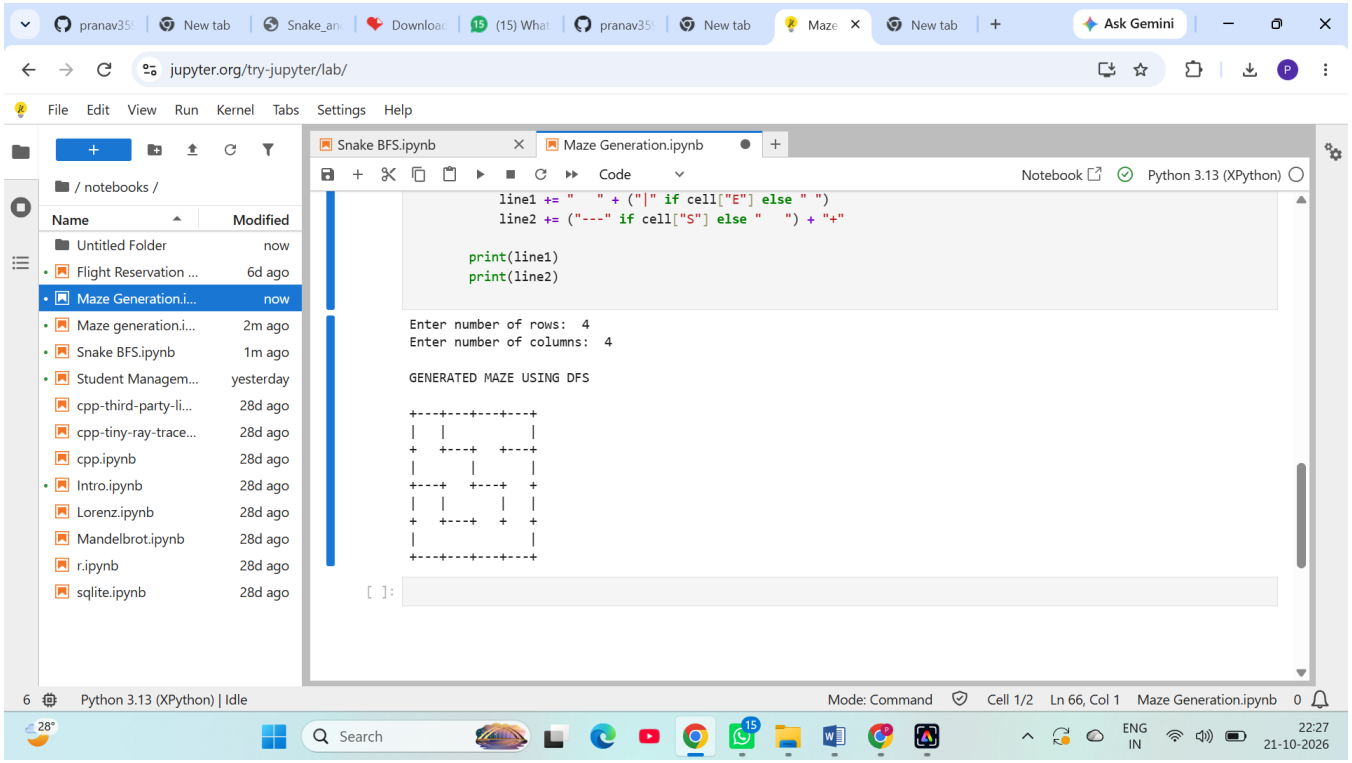


Fig 6.1: Result (Generated Maze using DFS)

7. APPLICATIONS

DFS-based maze generation is useful in several areas of computing and interactive applications. Typical applications include:

- Generating mazes for puzzle games and game levels.
- Creating randomized environments for pathfinding demonstrations and simulations.
- Analysing customer reviews, feedback and social media posts to find frequently discussed topics.
- Plagiarism detection and document similarity comparison.
- Teaching graph traversal, recursion, stacks and backtracking.
- Log analysis and counting the occurrence of events, errors or user actions.

8. FUTURE WORK

The project can be extended in the following ways:

- Add an interactive interface to adjust maze width, height and generation speed.
- Allow users to select different maze-generation algorithms and compare their output.
- Add a pathfinding algorithm to solve the generated maze after generation.
- Export generated mazes to image files or use them as levels in a simple game.
- Use a heap (heapq.nlargest) for top_n to reduce the sorting cost, and add a graphical word cloud of the results.

9. CONCLUSION

In this project, a maze was generated using Depth-First Search. The grid stores the wall information for each cell, while DFS visits unvisited neighbours, removes shared walls and backtracks when necessary. The process creates a connected maze in which every cell can be reached.

The program demonstrates the use of graph traversal, visited-state tracking, stacks and backtracking. The generated maze can be used for puzzle games, pathfinding demonstrations and further algorithm comparisons.

10. REFERENCES

[1] J. Vijaya et al., “Analysis of Maze Generation Algorithms,” in *2024 5th International Conference on Innovative Trends in Information Technology (ICITIIT)*, 2024. DOI: https://doi.org/10.1109/ICITIIT61487.2024.10580178

[2] J. Li, “Maze Generation Algorithms: A Comparative Research with a Focus on Recursive Backtracking in OCaml,” in *Proceedings of the 2nd International Conference on Engineering Management, Information Technology and Intelligence (EMITI 2025)*, pp. 432–437, 2025. DOI: https://doi.org/10.5220/0014360800004718

[3] J. Li, “Comprehensive Comparison of Maze Generating and Solving Algorithms,” in *Proceedings of the 3rd International Conference on Data Analysis and Machine Learning (DAML 2025)*, pp. 498–504, 2025. DOI: https://doi.org/10.5220/0014771700004818

[4] K. Godin-Dubois, K. Miras, and A. V. Kononova, “AMaze: An Intuitive Benchmark Generator for Fast Prototyping of Generalizable Agents,” *Frontiers in Artificial Intelligence*, vol. 8, article 1511712, 2025. DOI: https://doi.org/10.3389/frai.2025.1511712

[5] J. Kang, “A Comparative Analysis of DFS, Eller's and Wilson's Algorithms for Maze Generation,” *Highlights in Science, Engineering and Technology*, vol. 161, pp. 30–33, 2026. DOI: https://doi.org/10.54097/7zpwee86